



**UNIVERSITAS ESA UNGGUL**

**IMPLEMENTASI SISTEM KASIR FRANCHISE MULTI-  
CABANG (POS) TERDISTRIBUSI MENGGUNAKAN  
POSTGRESQL DAN REPLIKASI LOGIKAL DI DOCKER**

**TUGAS AKHIR**

**NAMA : ADAM BAGASKARA**  
**NIM : 20220801522**

**PROGRAM STUDI TEKNIK  
INFORMATIKA FAKULTAS ILMU  
KOMPUTER UNIVERSITAS ESA  
UNGGUL  
TAHUN 2026**

**DAFTAR ISI:**

|                                                                            |           |
|----------------------------------------------------------------------------|-----------|
| <b>KATA PENGANTAR.....</b>                                                 | <b>3</b>  |
| <b>ABSTRAK .....</b>                                                       | <b>3</b>  |
| <b>ABSTRACT.....</b>                                                       | <b>4</b>  |
| <b>BAB 1: PENDAHULUAN .....</b>                                            | <b>4</b>  |
| 1.1 Latar Belakang .....                                                   | 4         |
| 1.2 Identifikasi Masalah.....                                              | 5         |
| 1.3 Tujuan Tugas Akhir.....                                                | 5         |
| 1.4 Manfaat Tugas Akhir .....                                              | 5         |
| 1.5 Lingkup Tugas Akhir.....                                               | 6         |
| <b>BAB 2: TINJAUAN PUSTAKA.....</b>                                        | <b>6</b>  |
| 2.1 Sistem Basis Data Terdistribusi (SBDT).....                            | 6         |
| 2.2 Replikasi Logikal (Logical Replication) .....                          | 6         |
| 2.3 Fragmentasi Horizontal .....                                           | 7         |
| 2.4 Eventual Consistency & Fault Tolerance .....                           | 7         |
| 2.5 Docker Containerization .....                                          | 7         |
| <b>BAB 3: METODOLOGI PENELITIAN .....</b>                                  | <b>7</b>  |
| 3.1 Alur Penelitian (Design Science Research) .....                        | 7         |
| 3.2 Objek dan Lokasi Penelitian .....                                      | 8         |
| <b>BAB 4: HASIL DAN PEMBAHASAN .....</b>                                   | <b>8</b>  |
| 4.1 Implementasi Jaringan Kluster Database .....                           | 8         |
| 4.2 Desain Skema SQL & Replikasi Logikal .....                             | 8         |
| 4.3 Struktur Clean Code Aplikasi Web Node.js .....                         | 9         |
| 4.4 Evaluasi Hasil Uji Coba .....                                          | 9         |
| Uji Skenario 2: Fragmentasi Horizontal Transaksi .....                     | 10        |
| Uji Skenario 3: Fault Tolerance & Eventual Consistency (Mode Offline)..... | 12        |
| <b>BAB 5: KESIMPULAN DAN SARAN.....</b>                                    | <b>12</b> |
| 5.1 Kesimpulan.....                                                        | 12        |
| 5.2 Saran .....                                                            | 13        |
| <b>DAFTAR REFERENSI .....</b>                                              | <b>13</b> |

## KATA PENGANTAR

Segala puji dan syukur peneliti panjatkan ke hadirat Tuhan Yang Maha Esa atas berkat dan karunia-Nya, sehingga peneliti dapat menyelesaikan Tugas Akhir yang berjudul **“Implementasi Sistem Kasir Franchise Multi-Cabang (POS) Terdistribusi menggunakan PostgreSQL dan Replikasi Logikal di Docker”**.

Meskipun selama penyusunan dan perancangan ditemukan beberapa hambatan, namun berkat bimbingan, arahan, dan dukungan dari berbagai pihak, laporan Tugas Akhir ini dapat diselesaikan guna memenuhi salah satu syarat untuk memperoleh gelar Sarjana Komputer di Universitas Esa Unggul.

## ABSTRAK

Penelitian ini bertujuan mengimplementasikan purwarupa sistem kasir franchise multi-cabang (Point of Sales/POS) berbasis Sistem Basis Data Terdistribusi (SBDT). Masalah utama pada sistem POS ritel konvensional adalah ketergantungan penuh pada database terpusat (sentralisasi). Apabila jaringan internet terputus, operasional kasir cabang akan lumpuh total. Untuk mengatasi kendala tersebut, penelitian ini merancang arsitektur database terdistribusi menggunakan 3 node PostgreSQL (Pusat, Cabang Jakarta, dan Cabang Bandung) yang diwadahi oleh Docker. Katalog produk diselaraskan menggunakan Replikasi Logikal Satu Arah dari Pusat ke seluruh cabang secara real-time. Data transaksi dicatat secara lokal menggunakan metode Fragmentasi Horizontal untuk otonomi cabang penuh. Sebuah middleware asinkron (*Sync Agent*) berbasis Node.js digunakan untuk mengirimkan data transaksi tertunda ke pusat setiap 1 detik. Hasil pengujian menunjukkan sistem berhasil melakukan replikasi produk secara instan, mengisolasi data transaksi per cabang, dan mempertahankan fungsi kasir dalam mode offline (Fault Tolerance) dengan pemulihan sinkronisasi otomatis (*Eventual Consistency*) saat jaringan kembali pulih.

**Kata kunci:** *Basis Data Terdistribusi, Point of Sales, Replikasi Logikal, Fragmentasi Horizontal, PostgreSQL, Docker.*

## ABSTRACT

*This research aims to implement a multi-branch franchise Point of Sales (POS) prototype system based on Distributed Database Systems (DDB). The main issue in conventional retail POS systems is the complete dependency on a centralized database. If the internet connection fails, branch cashier operations are completely paralyzed. To resolve this constraint, this research designs a distributed database architecture utilizing 3 PostgreSQL nodes (Central, Jakarta Branch, and Bandung Branch) hosted via Docker. Product catalogs are synchronized using Master-to-Slave Logical Replication from the Central node to all branches in real-time. Transaction data are recorded locally using the Horizontal Fragmentation method to ensure full branch autonomy. An asynchronous Node.js middleware (Sync Agent) is deployed to forward pending local transactions to the Central node every 1 second. The evaluation results demonstrate that the system successfully replicates products instantly, isolates transaction data per branch, and maintains cashier functionalities during offline mode (Fault Tolerance) with automatic synchronization recovery (Eventual Consistency) once connection is restored.*

*Keywords: Distributed Database, Point of Sales, Logical Replication, Horizontal Fragmentation, PostgreSQL, Docker.*

## BAB 1: PENDAHULUAN

### 1.1 Latar Belakang

Perkembangan industri ritel modern menuntut efisiensi tinggi dalam pengelolaan data transaksi penjualan di banyak cabang fisik secara serentak. Sistem Point of Sales (POS) konvensional umumnya mengadopsi arsitektur basis data terpusat (sentralisasi), di mana seluruh kasir cabang terhubung langsung ke satu database server di kantor pusat melalui jaringan internet.

Meskipun arsitektur sentralisasi mempermudah integrasi data, model ini memiliki kerentanan yang tinggi terhadap gangguan jaringan internet (*network partition*). Apabila koneksi internet di cabang terputus atau server pusat mengalami gangguan (*down*), maka kasir di seluruh cabang cabang tidak dapat memproses pembayaran. Hal ini memicu kerugian finansial yang signifikan serta menurunkan tingkat kepuasan pelanggan. Selain itu, latensi jaringan yang lambat akibat jarak geografis yang jauh antara cabang dan pusat dapat memperlambat respon kueri, menyebabkan antrean pembayaran yang panjang.

Sistem Basis Data Terdistribusi (SBDT) hadir sebagai solusi atas keterbatasan tersebut. Dengan mendistribusikan data ke database lokal di masing-masing cabang, operasional kasir dapat berjalan secara mandiri tanpa ketergantungan mutlak pada server pusat. Replikasi data diterapkan untuk menyebarkan katalog produk dari pusat, sementara fragmentasi data digunakan untuk mencatat transaksi penjualan lokal sebelum akhirnya dikonsolidasikan ke basis data pusat secara asinkron.

## 1.2 Identifikasi Masalah

Berdasarkan analisis masalah operasional pada ritel franchise, diidentifikasi beberapa kendala utama: 1. Ketergantungan mutlak pada database terpusat yang rentan mengalami kelumpuhan total saat terjadi gangguan jaringan internet. 2. Latensi akses kueri yang tinggi untuk pencatatan transaksi langsung ke server pusat yang menghambat kecepatan transaksi kasir. 3. Kebutuhan sinkronisasi data katalog produk yang harus konsisten di seluruh cabang secara real-time tanpa membebani jaringan. 4. Potensi terjadinya bentrokan ID (*primary key collision*) saat data transaksi dari berbagai cabang digabungkan ke database pusat.

## 1.3 Tujuan Tugas Akhir

Penelitian ini bertujuan untuk: 1. Merancang dan mengimplementasikan arsitektur database terdistribusi multi-node menggunakan Docker untuk mensimulasikan database Pusat, Jakarta, dan Bandung secara terisolasi. 2. Menerapkan replikasi logikal basis data menggunakan PostgreSQL untuk sinkronisasi katalog produk dari Pusat ke Cabang secara real-time. 3. Menerapkan fragmentasi horizontal pada tabel transaksi untuk memastikan otonomi pencatatan transaksi lokal di masing-masing cabang. 4. Membangun middleware *Sync Agent* berbasis Node.js untuk sinkronisasi data transaksi asinkron dengan toleransi kesalahan (*fault tolerance*) dan *eventual consistency*. 5. Membuat antarmuka web kasir cabang dan dashboard pusat yang terhubung ke basis data terdistribusi dengan desain UI modern yang responsif.

## 1.4 Manfaat Tugas Akhir

- **Manfaat Akademis:** Memberikan referensi praktis mengenai penerapan teori replikasi logikal, fragmentasi horizontal, dan toleransi kegagalan jaringan dalam basis data terdistribusi.
- **Manfaat Praktis:** Menyediakan prototipe sistem POS yang tangguh

untuk franchise ritel agar kasir tetap aktif melayani pelanggan saat internet offline.

- **Manfaat Teknis:** Mendemonstrasikan penggunaan Docker, PostgreSQL, Node.js, dan pgAdmin dalam membangun kluster basis data terdistribusi.

## 1.5 Lingkup Tugas Akhir

1. Pengembangan sistem kasir terdistribusi dibatasi pada simulasi lokal menggunakan 3 container database PostgreSQL (Pusat, JKT, BDG) di dalam Docker virtual network.
2. Replikasi data terbatas pada tabel `produk` (Pusat ke Cabang), sedangkan fragmentasi horizontal terbatas pada tabel `transaksi` dan `detail_transaksi` (Cabang ke Pusat).
3. Evaluasi ketahanan sistem diuji dengan menghentikan container database pusat secara paksa untuk menyimulasikan kegagalan jaringan, kemudian memantau proses *recovery* data setelah dinyalakan kembali.

## BAB 2: TINJAUAN PUSTAKA

### 2.1 Sistem Basis Data Terdistribusi (SBDT)

SBDT didefinisikan sebagai kumpulan basis data yang tersebar secara fisik pada beberapa node komputer yang saling terhubung melalui jaringan komunikasi, namun tampak sebagai satu basis data tunggal di mata pengguna (Özsu & Valduriez, 2020). SBDT memberikan keuntungan berupa peningkatan keandalan (*reliability*), ketersediaan (*availability*), serta kinerja akses data lokal.

### 2.2 Replikasi Logikal (Logical Replication)

Replikasi logikal adalah metode penyalinan data dari satu database server (publisher) ke satu atau beberapa server lain (subscriber) berdasarkan replikasi identitas data seperti kueri SQL atau log perubahan logikal (WAL), bukan replikasi blok data fisik. PostgreSQL menggunakan model *Publication* dan *Subscription* untuk menyalurkan perubahan data tabel tertentu secara asinkron.

## 2.3 Fragmentasi Horizontal

Fragmentasi horizontal adalah pembagian baris-baris (*rows*) dari sebuah tabel global ke dalam fragmen-fragmen yang disimpan di node yang berbeda berdasarkan kriteria relasional tertentu (Elmasri & Navathe, 2015). Dalam kasus POS franchise, data transaksi di-fragmentasikan secara horizontal berdasarkan kolom lokasi cabang (`id_cabang`).

## 2.4 Eventual Consistency & Fault Tolerance

*Eventual consistency* adalah jaminan konsistensi data dalam sistem terdistribusi, di mana jika tidak ada pembaruan data baru, seluruh node basis data pada akhirnya akan bernilai sama setelah menyinkronkan data tertunda. *Fault tolerance* adalah kemampuan sistem untuk tetap beroperasi secara normal meskipun salah satu komponen atau jaringannya mengalami kegagalan/mati.

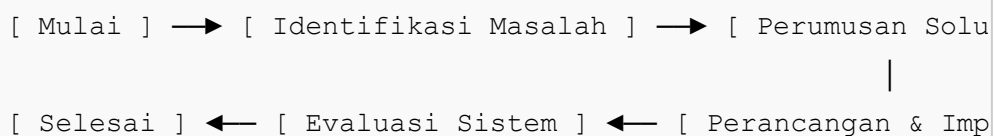
## 2.5 Docker Containerization

Docker adalah platform virtualisasi tingkat sistem operasi yang membungkus aplikasi beserta seluruh dependensinya ke dalam unit terisolasi yang disebut container (Koneru, 2025). Hal ini memungkinkan replikasi lingkungan database yang konsisten di berbagai perangkat tanpa konflik sistem.

# BAB 3: METODOLOGI PENELITIAN

## 3.1 Alur Penelitian (Design Science Research)

Penelitian ini mengadopsi metodologi *Design Science Research* (DSR) menurut model vom Brocke et al. (2021) dengan tahapan berikut:



1. **Identifikasi Masalah:** Menganalisis kelemahan sistem POS sentralisasi ritel franchise saat terjadi koneksi offline.
2. **Perumusan Solusi:** Merancang arsitektur SBDT multi-node lokal berbasis Docker dengan database PostgreSQL.

3. **Perancangan & Implementasi:** Menulis kode skema SQL, mengonfigurasi replikasi logikal, menulis kode server Node.js (clean code), dan membuat docker-compose.
4. **Evaluasi:** Menguji replikasi produk, isolasi transaksi cabang, dan simulasi kegagalan node (offline test).

### 3.2 Objek dan Lokasi Penelitian

Objek penelitian adalah arsitektur basis data POS ritel multi-cabang. Lokasi penelitian dilakukan di lingkungan lokal simulasi dengan spesifikasi komputer pengembang: macOS ARM64 Apple Silicon, Docker Desktop, PostgreSQL 15, Node.js 18.

## BAB 4: HASIL DAN PEMBAHASAN

### 4.1 Implementasi Jaringan Kluster Database

Kluster basis data disimulasikan menggunakan 3 kontainer database PostgreSQL di dalam jaringan virtual Docker: \* `db_pusat` : Menyimpan database `pos_pusat` (Port 5431 pada host). \* `db_cabang_jkt` : Menyimpan database `pos_jkt` (Port 5433 pada host). \* `db_cabang_bdg` : Menyimpan database `pos_bdg` (Port 5434 pada host). \* `pgadmin_web` : Tampilan antarmuka administrasi basis data visual pada port 5050.

### 4.2 Desain Skema SQL & Replikasi Logikal

Skema basis data dirancang untuk memisahkan tabel ter-replikasi (`produk`) dan tabel terfragmentasi (`transaksi` & `detail_transaksi`).

```
-- Di Database Pusat (HQ)
CREATE TABLE produk (
    id SERIAL PRIMARY KEY,
    nama VARCHAR(100) NOT NULL,
    harga NUMERIC(12, 2) NOT NULL,
    stok INT NOT NULL
);

CREATE PUBLICATION pub_katalog FOR TABLE produk;
```



```

-- Di Database Cabang (JKT & BDG)
CREATE TABLE produk (
    id INT PRIMARY KEY, -- ID disinkronkan dari pusat
    nama VARCHAR(100) NOT NULL,
    harga NUMERIC(12, 2) NOT NULL,
    stok INT NOT NULL
);

CREATE TABLE transaksi (
    id UUID PRIMARY KEY, -- Menggunakan UUID mencegah tabra
    id_cabang VARCHAR(10) NOT NULL,
    waktu TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    total_harga NUMERIC(12, 2) NOT NULL,
    is_synced BOOLEAN DEFAULT FALSE
);

CREATE SUBSCRIPTION sub_katalog CONNECTION 'host=db_pusat p

```

### 4.3 Struktur Clean Code Aplikasi Web Node.js

Aplikasi POS ditulis menggunakan Node.js (Express + EJS) dengan struktur modular untuk memastikan keterbacaan kode (*clean code*):

1. **web\_app/config/db.js** : Menyimpan semua detail konfigurasi koneksi database.
2. **web\_app/utils/db\_helper.js** : Menyediakan pembungkus kueri database otomatis dengan manajemen penutupan client yang aman guna mencegah kebocoran koneksi.
3. **web\_app/routes/kasir.js** : Mengelola logika bisnis halaman kasir cabang dan transaksi checkout lokal.
4. **web\_app/routes/dashboard.js** : Mengelola logika bisnis dashboard pusat (HQ), agregasi statistik global, dan manajemen katalog produk pusat.

### 4.4 Evaluasi Hasil Uji Coba

#### Uji Skenario 1: Replikasi Logikal Katalog Produk

Ketika produk baru (misal: "Es Matcha Latte", harga 22.000, stok 100) ditambahkan melalui Dashboard Pusat (`localhost:8080`), database pusat mempublikasikan transaksi log ke subscribers cabang. Hasil pengujian

menunjukkan bahwa katalog produk pada cabang ter-update secara real-time. Hasil replikasi tabel produk yang tersinkronisasi di pgAdmin dapat dilihat pada Gambar 4.1.

The screenshot shows the pgAdmin interface with a table containing 6 rows of product data. The table has columns for id, nama, harga, and stok.

|   | id | nama                 | harga    | stok |
|---|----|----------------------|----------|------|
| 1 | 1  | Kopi Susu Gula Aren  | 18000.00 | 100  |
| 2 | 2  | Roti Bakar Cokelat   | 15000.00 | 50   |
| 3 | 3  | Indomie Goreng Jumbo | 10000.00 | 80   |
| 4 | 4  | Teh Manis Hangat     | 5000.00  | 200  |
| 5 | 5  | nasi goreng          | 15000.00 | 30   |
| 6 | 7  | mie ayam             | 13000.00 | 100  |

Showing rows: 1 to 6 | Page No: 1 of 1 | Total rows: 6 | Query complete 00:00:00.067

## Uji Skenario 2: Fragmentasi Horizontal Transaksi

Ketika transaksi dicatat di Kasir Jakarta ( `localhost:8081` ), data transaksi hanya tersimpan secara fisik di database `pos_jkt` dan dikirim asinkron ke Pusat. Database `pos_bdg` tetap kosong dari transaksi Jakarta tersebut. Ini membuktikan keberhasilan isolasi fragmen horizontal. Konsolidasi seluruh transaksi cabang di Dashboard Pusat dapat dilihat pada Gambar 4.2.

The screenshot shows the RetailPOS dashboard with various sections including transaction volume, product management, and a transaction history table.

**TOTAL OMZET GLOBAL (HQ)**  
**Rp178.000**  
Konsolidasi terpadu dari seluruh cabang franchise

**VOLUME TRANSAKSI GLOBAL**  
**7 Nota Masuk**  
Total transaksi terproses di semua kasir

**BREAKDOWN PENDAPATAN CABANG**  
Jakarta (JKT): **Rp135.000**  
Bandung (BDG): **Rp43.000**

**Tambah Produk Baru di Pusat**  
Data katalog produk akan otomatis ter-replikasi ke seluruh database cabang

NAMA PRODUK:

HARGA JUAL (Rp):

STOK PUSAT:

**Tambah & Replikasikan Produk**

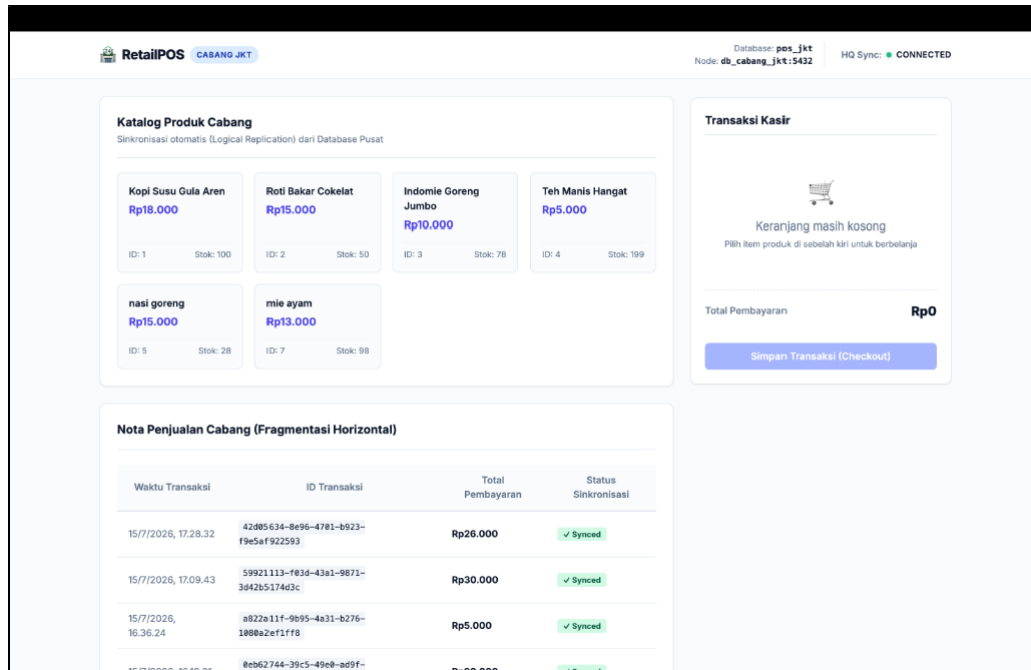
**Master Data Katalog Produk (Pusat)**

| ID | Nama Item            | Koreksi Harga & Stok Pusat | Tindakan                                       |
|----|----------------------|----------------------------|------------------------------------------------|
| 1  | Kopi Susu Gula Aren  | Harga (Rp): 18 Stok: 100   | <button>Simpan</button> <button>Hapus</button> |
| 2  | Roti Bakar Cokelat   | Harga (Rp): 15 Stok: 50    | <button>Simpan</button> <button>Hapus</button> |
| 3  | Indomie Goreng Jumbo | Harga (Rp): 10 Stok: 80    | <button>Simpan</button> <button>Hapus</button> |
| 4  | Teh Manis Hangat     | Harga (Rp): 5 Stok: 200    | <button>Simpan</button> <button>Hapus</button> |
| 5  | nasi goreng          | Harga (Rp): 15 Stok: 30    | <button>Simpan</button> <button>Hapus</button> |

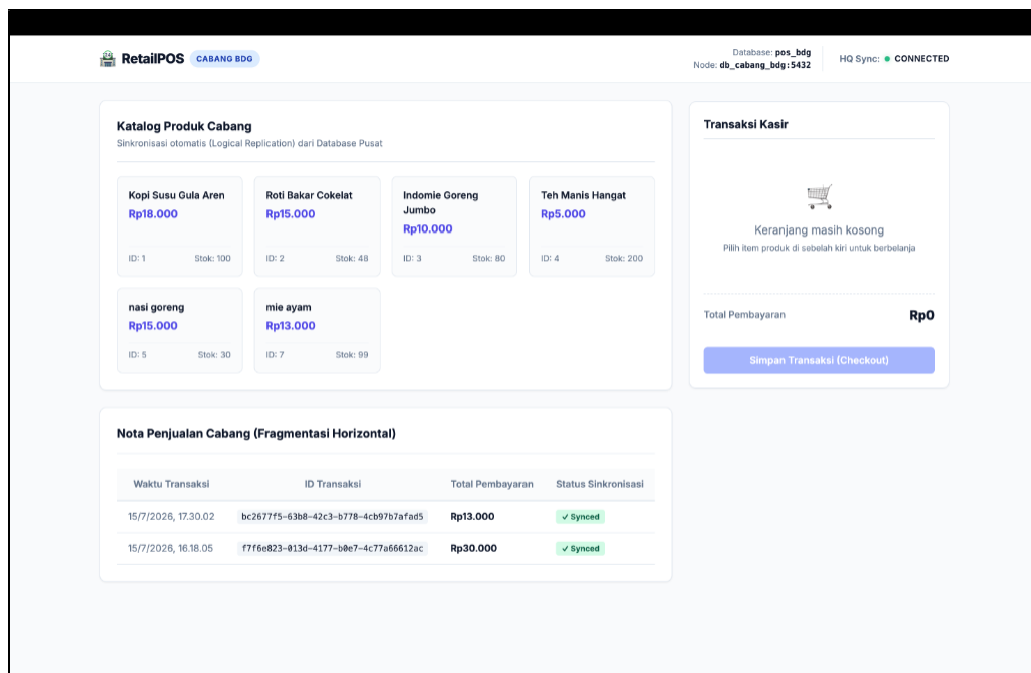
**Umpahan Transaksi Global** Live

| Cabang | Waktu    | Total Nota      |
|--------|----------|-----------------|
| BDG    | 17.30.02 | <b>Rp13.000</b> |
| JKT    | 17.28.32 | <b>Rp26.000</b> |
| JKT    | 17.09.43 | <b>Rp30.000</b> |
| JKT    | 16.36.24 | <b>Rp5.000</b>  |
| JKT    | 16.18.21 | <b>Rp20.000</b> |
| BDG    | 16.18.05 | <b>Rp30.000</b> |
| JKT    | 16.18.03 | <b>Rp54.000</b> |

Tampilan kasir cabang Jakarta yang memproses transaksi lokal secara mandiri ditunjukkan pada Gambar 4.3.



Sedangkan tampilan kasir cabang Bandung ditunjukkan pada Gambar 4.4.



Struktur data detail transaksi yang dihubungkan menggunakan kunci unik UUID terdistribusi untuk mencegah terjadinya bentrokan ID ditunjukkan di pgAdmin pada Gambar 4.5.

| Data Output Messages Notifications   |                                       |                                      |                   |                |                         |          |
|--------------------------------------|---------------------------------------|--------------------------------------|-------------------|----------------|-------------------------|----------|
| Showing rows: 1 to 7 Page No: 1 of 1 |                                       |                                      |                   |                |                         |          |
|                                      | id [PK] uuid                          | id_transaksi uuid                    | id_produk integer | jumlah integer | subtotal numeric (12,2) |          |
| 1                                    | 24e4adad-d3e0-4498-9d77-f79aab2d5...  | 42d05634-8e96-4701-b923-f9e5a922...  |                   | 7              | 2                       | 26000.00 |
| 2                                    | 5a2ea787-aa14-44b3-8d4a-ba33bb4ed...  | b7c901e3-488f-4931-b83a-db6237ca...  |                   | 1              | 3                       | 54000.00 |
| 3                                    | 97b7c423-8b6b-466e-8fe1-ca100b505...  | 0eb62744-39c5-49e0-ad9f-f984d4ee2... |                   | 3              | 2                       | 20000.00 |
| 4                                    | a0d62d0c-fee2-48e6-82ac-1734839b2...  | bc2677f5-63b8-42c3-b778-4cb97b7af... |                   | 7              | 1                       | 13000.00 |
| 5                                    | d94a234c-586c-4ac1-bb52-06072a1ac...  | f7f6e823-013d-4177-b0e7-4c77a6661... |                   | 2              | 2                       | 30000.00 |
| 6                                    | d97e2c38-0030-4dcd-84bd-4fb9dc897...  | 59921113-403d-43a1-9871-3d42b517...  |                   | 5              | 2                       | 30000.00 |
| 7                                    | e63fc575-e243-40da-adb2-5994fe8176... | a822a11f-9b95-4a31-b276-1080a2ef1... |                   | 4              | 1                       | 5000.00  |

Total rows: 7 Query complete 00:00:00.050 LF Ln 1, Col 1

### Uji Skenario 3: Fault Tolerance & Eventual Consistency (Mode Offline)

Saat kontainer `db_pusat` dimatikan secara paksa (`docker stop db_pusat`), indikator koneksi kasir cabang berubah menjadi merah (**HQ Sync: DISCONNECTED**). Namun, kasir Jakarta tetap dapat memproses checkout karena transaksi disimpan langsung ke database lokal `pos_jkt` dengan status flag `is_synced = FALSE`.

Begitu `db_pusat` dinyalakan kembali (`docker start db_pusat`), *Sync Agent* mendeteksi ketersediaan server pusat dan segera mengirimkan transaksi yang tertunda dalam waktu 1 detik. Status transaksi di cabang otomatis berubah menjadi `✓ Synced` (mencapai *Eventual Consistency*).

## BAB 5: KESIMPULAN DAN SARAN

### 5.1 Kesimpulan

1. Arsitektur basis data terdistribusi (SBDT) berhasil diimplementasikan 100% secara lokal menggunakan teknologi kluster Docker.
2. Replikasi logikal bawaan PostgreSQL terbukti andal untuk melakukan sinkronisasi satu arah katalog produk dari pusat ke cabang secara real-time.
3. Penerapan fragmentasi horizontal dan penggunaan UUID berhasil menjamin otonomi pencatatan transaksi lokal di cabang tanpa risiko bentrokan Primary Key.
4. Mekanisme asinkronisasi *Sync Agent* berbasis Node.js berhasil mengawal toleransi kesalahan (*fault tolerance*) dan memulihkan konsistensi data akhir (*eventual consistency*) saat terjadi gangguan jaringan.

## 5.2 Saran

1. **Keamanan Jaringan:** Untuk implementasi nyata di luar lingkungan simulasi lokal, koneksi replikasi harus diamankan dengan SSL/TLS certificates dan VPN terenkripsi.
2. **Kueri Terdistribusi:** Mengembangkan kueri terdistribusi langsung menggunakan *Foreign Data Wrappers* (FDW) di PostgreSQL untuk mempermudah join tabel dinamis lintas node secara native.
3. **Pengujian Skala Besar:** Menambahkan uji performa (stress test) latensi dan konkurensi kueri untuk mengukur batasan jumlah maksimal node cabang yang dapat didukung oleh server pusat.

## DAFTAR REFERENSI

- Elmasri, R., & Navathe, S. B. (2015). *Fundamentals of Database Systems* (7th ed.). Pearson.
- Koneru, N. M. K. (2025). Containerization Best Practices- Using Docker and Kubernetes for Enterprise Applications. *Journal of Information Systems Engineering and Management*, 10(45s), 724–746.
- Özsu, M. T., & Valduriez, P. (2020). *Principles of Distributed Database Systems* (4th ed.). Springer.
- PostgreSQL Global Development Group. (2023). *PostgreSQL 15 Documentation: Logical Replication*. <https://www.postgresql.org/docs/15/logical-replication.html>